

Doc. no. P0003R1
Date: 2016-02-09
Project: Programming Language C++
Audience: Core Working Group
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Removing Deprecated Exception Specifications from C++17

Table of Contents

- [Introduction](#)
- [A Brief History of Exception Specifications](#)
- [Proposed Changes](#)
- [Motivation for Change](#)
- [Compatibility Concerns](#)
- [Review](#)
 - [Evolution Working Group Review](#)
 - [Design Choices](#)
 - [Interaction with issues](#)
 - [Drafting Choices](#)
- [Proposed Wording](#)
 - [First cut, using current set-based wording](#)
 - [4.12 Function pointer conversions \[conv.fctptr\]](#)
 - [8.3.5 Functions \[dcl.fct\]](#)
 - [8.4.2 Explicitly-defaulted functions \[dcl.fct.def.default\]](#)
 - [14.5.3 Variadic templates \[temp.variadic\]](#)
 - [15.3 Handling an exception \[except.handle\]](#)
 - [15.4 Exception specifications \[except.spec\]](#)
 - [15.5 Special functions \[except.special\]](#)
 - [15.5.1 The `std::terminate\(\)` function \[except.terminate\]](#)
 - [15.5.2 The `std::unexpected\(\)` function \[except.unexpected\]](#)
 - [17.6.4.7 Handler functions \[handler.functions\]](#)
 - [17.6.4.8 Other functions \[res.on.functions\]](#)
 - [17.6.5.12 Restrictions on exception handling \[res.on.exception.handling\]](#)
 - [18.8 Exception Handling \[support.exception\]](#)
 - [Header `<exception>` synopsis](#)
 - [18.8.3 Class `bad\_exception` \[bad.exception\]](#)
 - [23.3.7.8 Zero sized arrays \[array.zero\]](#)
 - [Annex B \(informative\) Implementation quantities \[implimits\]](#)
 - [C.4.x Clause 15: declarations \[diff.cpp14.dcl.dcl\]](#)
 - [D.2 Dynamic exception specifications \[depr.except.spec\]](#)
 - [D.5 Violating *exception-specifications* \[exception.unexpected\]](#)
 - [D.5.1 Type `unexpected\_handler` \[unexpected.handler\]](#)
 - [D.5.2 `set\_unexpected` \[set.unexpected\]](#)
 - [D.5.3 `get\_unexpected` \[get.unexpected\]](#)
 - [D.5.4 `unexpected` \[unexpected\]](#)
 - [Second cut: simplified wording](#)
 - [5.3.7 noexcept operator \[expr.unary.noexcept\]](#)
 - [8.4.2 Explicitly-defaulted functions \[dcl.fct.def.default\]](#)
 - [15.4 Exception specifications \[except.spec\]](#)
- [Acknowledgements](#)
- [References](#)

Revision History

Revision 0

Original version of the paper for the 2015 pre-Kona mailing.

Revision 1

Revised the wording to modify the proposed working draft after the Kona 2015 meeting, N4567. This accommodates renumbering a few clauses, notably Annex D where several other features have already been removed, and allowing for exception specifications in the type system, and the rewording of inheriting constructors.

Introduction

Dynamic exception specifications were deprecated in C++11. This paper formally proposes removing the feature from C++17, while retaining the (still) deprecated `throw()` specification strictly as an alias for `noexcept(true)`.

A Brief History of Exception Specifications

Exception specifications were added as part of the original design of the exception language feature. However, the experience of using them was less than desired, with the general consensus by the time the 1998 standard was published being, generally, to not use them. The standard library made the explicit choice to not use exception specifications apart from a handful of places where guaranteeing an empty (no-throwing) specification seemed useful. [N741](#) gives a brief summary of the LWG thoughts at the time.

By the time C++11 was published, the feeling against exception specifications had grown, the limited real-world use generally reported negative user experience, and the language feature, renamed as *dynamic* exception specifications, was deprecated, ([N3051](#)). A new language feature, `noexcept`, was introduced to describe the important case of knowing when a function could guarantee to not throw any exceptions.

Looking ahead to C++17, there is a desire to incorporate exception specifications into the type system, [N4533](#). This solves a number of awkward corners that arise from exception specifications not being part of the type of a function, but still does nothing for the case of deprecated dynamic exception specifications, so the actual language does not get simpler, and we must still document and teach the awkward corners that arise from dynamic exception specifications outside the type system.

Proposed Changes

The recommendation of this paper is to remove dynamic exception specifications from the language. However, the syntax of the `throw()` specification should be retained, but no longer as a dynamic exception specification. Its meaning should become strictly an alias for `noexcept(true)`, and its usage should remain deprecated.

To minimize the impact on the current standard wording, the grammar term *exception-specification* is retained, although it has only one production and could be replaced entirely with *noexcept-specification*. Alternatively, the grammar term *noexcept-specification* could be retired.

The wording changes in this initial proposal are deliberately minimal in an effort to ensure the least risk of accidental change. However, rather than using the language of sets of compatible exception specifications (where there are now only two such sets, the empty set and the set of all types) it would be possible to write a simpler, clearer form describing instead functions that permit exceptions, and functions that permit *no* exceptions. While such a specification would be preferred, it is also beyond the drafting skills of the proposal author.

The redrafting goes slightly beyond minimal by eliminating use of the pseudo-type "any". This change, while improving clarity, also avoids confusion with the standard library type `any` from the Library Fundamentals TS in the event that it becomes part of a future standard library.

Motivation for Change

Dynamic exception specifications are a failed experiment, but this is not immediately clear to novices, especially where the "novice" is an experienced developer coming from other languages such as Java, where exception specifications may be more widely used. Their continuing presence, especially in an important part of the language model (exceptions are key to understanding constructors, destructors, and RAII) is an impediment that must be explained. It remains embarrassing to explain to new developers that there are features of the language that conforming compilers are required to support, yet should never be used.

Exception specifications in general occupy an awkward corner of the grammar, where they do not affect the type system, yet critically affect how a virtual function can be overridden, or which functions can bind to certain function pointer variables. As noted above, [N4533](#) would go a long way to resolving that problem for `noexcept` exception specifications, which makes the hole left for dynamic exception specifications even more awkward and unusual for the next generation of C++ developers.

C++17 is on schedule to be a release with several breaking changes for old code, with the standard library removing `auto_ptr`, the classic C++98 fuction binders, and more. Similarly, it is expected that the core language will lose trigraphs, the `register` keyword, and the increment operator for `bool`. This would be a good time to make a clean break with long discouraged (and actively deprecated) parts of the language.

The proposed change would resolve core issue [596](#) as no longer relevant (NAD), and should simplify core issue [1351](#), although that is marked as a Defect Report that was not yet applied to the working paper the proposed wording below was drafted from.

Compatibility Concerns

There is certainly some body of existing code that has not heeded the existing best practice of discouraging dynamic exception specifications, and has not yet accounted for the feature being deprecated in C++11. It is not clear how much of such code would be expected to port unmodified into a C++17 (or beyond) world, and the change is relatively simple - just strike the (non-empty) dynamic exception specification to retain equivalent meaning. The key difference is that the `unexpected` handler will not now be called to translate unexpected exceptions. In rare cases, this would allow a new exception type to propage, rather than calling `terminate` to abort. If enforcing that semantic is seen as important in production systems, there is a more intrusive workaround available:

```
void legacy() throw(something)
try
{
    // function body as before
}
catch(const something&) {
    throw;
}
catch(...) {
    terminate();
}
```

It is thought that the empty dynamic exception specification, `throw()`, was much more widely used in practice, often in the mistaken impression that compilers would use this information to optimize code generation where no exception could propagate, where in fact this is strictly a pessimization that forces stack unwinding to the function exit, and then calling the `unexpected` callback in a manner that is guatanteed to fail before the subsequent call to `terminate`. This paper proposes treating such (still deprecate) exception specifications as synonyms for `noexcept(true)`, yielding the performance benefit many were originally expecting.

It should also be noted that at least one widely distributed compiler has still not implemented this feature in 2015, and at least one vendor has expressed a desire to never implement the deprecated feature (while that vendor *has* implemented the `noexcept` form of exception specification). Code on that platform would not be adversely impacted by the proposed removal, and portable code must always have allowed for the idiosynracies of this platform.

One remaining task is to survey popular open source libraries and see what level of usage, if any, remains in large, easily accessible codebases.

Review

Evolution Working Group Review

The first version of this paper was presented to the Evolution Working Group at the Kona 2015 meeting, and was recommended to advance to Core 'as written'. Specifically, that means retaining the empty exception specification as an alternate spelling of `noexcept(true)`:

SF WF N WA SA
12 13 6 0 0

Design Choices

By choosing to keep the `throw()` specification, we look to retain source compatibility with a large body of existing code that has used this specification to indicate code that should not throw. However, we also change the meaning of that specification to be exactly the same as a plain `noexcept` specification, which means that an implementation is no longer permitted to call `unexpected` when a violation is detected (which was required in all previous standards) and the previously mnadated stack unwinding might no longer occur. This means that a conforming C++17 implementation is not permitted to simply retain the C++14 semantics while issuing an "extension" diagnostic.

Interaction with issues

- Core issue 596: (Open) NAD as the subclause is eliminated
- Core issue 1604: (wrong issue number, lost my reference?)
- Core issue 2039: (Tentatively Ready) Proposed resolution should apply on top of this wording
- Core issue 2047: (Ready) Proposed resolution incorporated into wording
- Core issue 2183: (Open) Must be resolved in this paper's re-wording
-
- Core issue 1912: (Extension) No direct impact
- Core issue 1934: (Extension) Redraft issue as underlying wording changes significantly

Drafting Choices

As the author is not a regular drafter of Core wording, a quick rationale for some of the most obvious wording decisions seems helpful, especially when drawing attention to changes that were deliberately *not* made.

Where there is some question over these decisions, it would probably be more efficient to have direct feedback here, than sowing repeating detailed comments throughout the wording - at least for the first round of review.

Retain the grammar production *exception-specification*

To minimize the impact on the existing grammar and wording, the term *exception-specification* is retained at exactly the same place in the grammar. However, it loses its ability to describe anything other than a *noexcept-specification*. The empty throw specification becomes a (deprecated) synonym for `noexcept(true)`, and the *noexcept-specification* production is retired.

An alternative formulation might have been to retire the original *exception-specification* itself, forcing us to find and address each existing use in the text - either to simply replace it with *noexcept-specification*, or rewrite that part of the document to no longer refer to exception specifications at all.

The author has gone with the first option as it will be a less disruptive set of changes, and the Core working group has the expertise to spot and remove any remaining redundancies as a later clean-up (if desired).

Retain the specification in terms of sets of types

The current text regarding exception specifications is written in terms of sets of types that might pass through the specification. This drafting removes the notion of an "any" type, and simplifies the world down to two sets: the empty set, and the set of all types.

The wording could be simplified and made clear by simply talking about specifications that permit exceptions to pass, and those that do not, and simply skip the introduction of a simple set algebra. However, that would be a much larger rewrite that should not be undertaken by a mere amateur in the world of Core drafting, so is left as an exercise for the Core Working Group itself, should it feel strongly motivated. It is also the kind of clean-up that can occur at a more deliberate pace after the initial proposal is adopted.

With the removal of dynamic exception specifications, why not remove the text for unwinding when an specification is violated?

The simple answer is that would be a technical change that needs to be separately blessed by EWG. The wording demanding unwinding at a violated exception specification can be cleanly removed with the whole of sub-clause 15.5.2. However, the implementation-defined choice on whether to unwind through an *exception-specification* should remain, to support the variety of exception unwinders deployed in production compilers today.

Do not touch wording that refers to types in exception specifications

There are a few clauses that talk about types when used in exception specifications, such as describing the name look-up rules, or requirements for type-completeness. With the removal of *dynamic-exception-specifications*, the first attempt at drafting updated these clauses too (and eagle-eyed readers may notice comments in the HTML source with that discarded drafting). However, in most or all cases, the existing text remains valid, but more specialized. There is still the use case of an exception specification using a `noexcept` operator, which would still rely on this wording for expressions within that operator.

For reference (and review by the diligent) the list of clauses affected by this decision to make no changes is:

- 3.3.7 p1 bullet 1 [basic.scope.class]
- 3.4.1 p7,8, footnote 31 [basic.lookup.unqual]

- 9.2 p2,3,4 [class.mem]
- 14.5 p2 [temp.decls]
- 14.6 p11 [temp.res]
- 14.6.4.1 p3 [temp.point]
- 14.7.1 p1,15 [temp.inst]
- 14.7.2 p12 [temp.explicit]
- 14.8.2 p7 [temp.deduct]

exception specification vs. exception-specification

Sometimes the wording refers to an *exception specification*, and at other times it refers to an *exception-specification*. This is generally intentional, where the former refers to the notion of an exception specification, which includes implicit exception specifications, where the latter refers to explicit use of the grammar production.

Open Questions

Where is the update for Compatibility Annex C?

Oops - remember to include `throw()` specifications might no longer unwind, but were required to unwind in C++14.

Is 15.4p5 redundant when combined with exception specifications in the type system?

It should be possible to greatly simplify this paragraph, if not eliminate it entirely.

Are there other relevant references that have been missed?

Proposed Wording

Two attempts are made at drafting new wording.

The first stays as close as possible to the original wording, retaining the idea of sets of types that may pass through an exception specification, but simplifying the idea so that only two sets exist in practice: the empty set, and the set of all types. The intent is to provide a basis for stable wording that can be adopted with confidence, at the earliest opportunity.

The second attempts to simplify the notion of exception specifications to those that allow exceptions, and those that do not allow exceptions. This removes the notion of set algebra from the standard, and talks much more directly to the topic of propagating exceptions. It is more of a rewrite though, and might be considered riskier as a consequence, even if it would be the preferred long term approach.

First cut, using current set-based wording

4.12 Function pointer conversions [conv.fctptr]

1 A prvalue of type “pointer to `noexcept` function” can be converted to a prvalue of type “pointer to function”. The result is a pointer to the function. A prvalue of type “pointer to member of type `noexcept` function” can be converted to a prvalue of type “pointer to member of type function”. The result points to the member function. [*Example*:

```
void (*p)() throw(int);
void (**pp)() noexcept = &p; // error: cannot convert to pointer to noexcept function
```

```
struct S { typedef void (*p)(); operator p(); };
void (*q)() noexcept = S(); // error: cannot convert to pointer to noexcept function
```

— *end example*]

8.3.5 Functions [dcl.fct]

8 The return type, the *parameter-type-list*, the *ref-qualifier*, the *cv-qualifier-seq*, and whether the function has a non-throwing *exception-specification*, but not the default arguments (8.3.6) ~~or the exception-specification (15.4)~~, are part of the function type. [Note: Function types are checked during the assignments and initializations of pointers to functions, references to functions, and pointers to member functions. — *end note*]

8.4.2 Explicitly-defaulted functions [dcl.fct.def.default]

4 [*Example*:

```

struct S {
    constexpr S() = default;           // ill-formed: implicit S() is not constexpr
    S(int a = 0) = default;           // ill-formed: default argument
    void operator=(const S&) = default; // ill-formed: non-matching return type
    ~S() noexcept(false) throw(int) = default; // deleted: exception specification does not match
private:
    int i;
    S(S&);                             // OK: private copy constructor
};
S::S(S&) = default;                   // OK: defines copy constructor

```

— end example]

14.5.3 Variadic templates [temp.variadic]

(4.7)—In a *dynamic-exception-specification* (15.4), the pattern is a *type-id*.

15.3 Handling an exception [except.handle]

7 A handler is considered active when initialization is complete for the parameter (if any) of the catch clause. [*Note*: The stack will have been unwound at that point. — end note] Also, an implicit handler is considered active when `std::terminate()` or `std::unexpected()` is entered due to a throw. A handler is no longer considered active when the catch clause exits or when `std::unexpected()` exits after being entered due to a throw.

15.4 Exception specifications [except.spec]

1 The *exception specification* of a function is a (possibly empty) set of types, indicating that the function might exit via an exception that matches a handler of one of the types in the set; the (conceptual) set of all types is used to denote that the function might exit via an exception of arbitrary type. If the set is empty, the function is said to have a *non-throwing exception specification*. The exception specification is either defined explicitly by using an *exception-specification* as a suffix of a function declaration's declarator (8.3.5) or implicitly.

```

exception-specification:
    dynamic-exception-specification
    noexcept-specification
    noexcept ( constant-expression )
    noexcept
    throw ( _ )

```

```

dynamic-exception-specification:
    throw ( type-id-listopt )

```

```

type-id-list:
    type-id ...opt
    type-id-list, type-id ...opt

```

```

noexcept-specification:
    noexcept ( constant-expression )
    noexcept

```

In an *exception* ~~noexcept~~-specification, the *constant-expression*, if supplied, shall be a constant expression (5.20) that is contextually converted to `bool` (Clause 4). A (token that follows `noexcept` is part of the *exception* ~~noexcept~~-specification and does not commence an initializer (8.5). The *exception-specification* `throw()` is deprecated (see Annex D), and equivalent to the *exception-specification* `noexcept`.

2 A type denoted in a *dynamic-exception-specification* shall not denote an incomplete type or an rvalue reference type. A type denoted in a *dynamic-exception-specification* shall not denote a pointer or reference to an incomplete type, other than `std::cv void*`. A type `cv T` denoted in a *dynamic-exception-specification* is adjusted to type `T`. A type `array of T`, or function type `T` denoted in a *dynamic-exception-specification* is adjusted to type `pointer to T`. A *dynamic-exception-specification* denotes an exception specification that is the set of adjusted types specified thereby.

3 The *exception-specification* `noexcept` or `noexcept ( constant-expression )`, where the *constant-expression* yields `true`, denotes an exception specification that is the empty set. The *exception-specification* `noexcept ( constant-expression )`, where the *constant-expression* yields `false`, or the absence of an *exception-specification* in a function declarator other than that for a destructor (12.4) or a deallocation function (3.7.4.2) denotes an exception specification that is the set of all types.

4 Two *exception-specifications* are *compatible* if the sets of types they denote are the same.

5 If any declaration of a function has an *exception-specification* that is not a *noexcept-specification* allowing `noall` exceptions, all declarations, including the definition and any explicit specialization, of that function shall have a compatible *exception-*

specification. If any declaration of a pointer to function, reference to function, or pointer to member function has an *exception-specification*, all occurrences of that declaration shall have a compatible *exception-specification*. If a declaration of a function has an implicit exception specification, other declarations of the function shall not specify an *exception-specification*. In an explicit instantiation an *exception-specification* may be specified, but is not required. If an *exception-specification* is specified in an explicit instantiation directive, it shall be compatible with the *exception-specifications* of other declarations of that function. A diagnostic is required only if the *exception-specifications* are not compatible within a single translation unit.

6 If a virtual function has an exception specification, all declarations, including the definition, of any function that overrides that virtual function in any derived class shall only allow exceptions that are allowed by the exception specification of the base class virtual function, unless the overriding function is defined as deleted. [*Example*:

```
struct B {
    virtual void f() noexceptthrow(int, double);
    virtual void g();
};

struct D: B {
    void f();                // ill-formed
    void g() noexceptthrow(int);    // OK
};
```

The declaration of D::f is ill-formed because it allows all exceptions, whereas B::f allows no exceptions~~only int and double~~. — *end example*]

~~7 An exception-specification can include the same type more than once and can include classes that are related by inheritance, even though doing so is redundant. [Note: An exception-specification can also include the class std::bad_exception (18.8.3). — end note]~~

~~8 A function is said to allow an exception of type E if its exception specification contains a type T for which a handler of type T would be a match (15.3) for an exception of type E. A function is said to allow all exceptions if its exception specification is the set of all types. Otherwise a function does not allow any exceptions.~~

9 Whenever an exception ~~of type E~~ is thrown and the search for a handler (15.3) encounters the outermost block of a function with an exception specification that does not allow any exceptions~~E~~, then, the function std::terminate() is called (15.5.1).

~~(9.1) — if the function definition has a dynamic-exception-specification, the function std::unexpected() is called (15.5.2);~~

~~(9.2) — otherwise, the function std::terminate() is called (15.5.1).~~

[*Example*:

```
class X {};
class Y {};
class Z: public X {};
class W {};

void f() throw(X, Y){
    int n = 0;
    if (n) throw X(); // OK
    if (n) throw Z(); // also OK
    throw W(); // will call std::unexpected()
}
```

— *end example*]

~~[Note: A function can have multiple declarations with different non-throwing exception-specifications; for this purpose, the one on the function definition is used. — end note]~~

10 An implementation shall not reject an expression merely because when executed it throws or might throw an exception that the containing function does not allow. [*Example*:

```
extern void f() throw(X, Y)noexcept;
void g() throw(X) {
    f();    // OK
}
```

the call to f is well-formed even though when called, f might throw an exception Y that g does not allow. — *end example*]

~~11 [Note: An exception specification is not considered part of a function's type; see 8.3.5. — end note]~~

~~12 A~~The set of potential exceptions of a given context is either a type or a set of types that might be thrown as an exception ~~or a pseudo-type, denoted by "any", that represents the situation where an exception of an arbitrary type might be thrown; the~~

(conceptual) set of all types is used to denote that an exception of arbitrary type might be thrown. A subexpression e_1 of an expression e is an immediate subexpression if there is no subexpression e_2 of e such that e_1 is a subexpression of e_2 .

13 The set of potential exceptions of a function, function pointer, or member function pointer f is defined as follows:

(13.1) — If the exception specification of f is the set of all types, the set consists of the pseudo-type "any".

(13.2) — Otherwise, the set consists of every type in the exception specification of f .

14 The set of potential exceptions of an expression e is empty if e is a core constant expression (5.20). Otherwise, it is the union of the sets of potential exceptions of the immediate subexpressions of e , including default argument expressions used in a function call, combined with a set s defined by the form of e , as follows:

(14.1) — If e is a function call (5.2.2):

(14.1.1) — If its *postfix-expression* is a (possibly parenthesized) *id-expression* (5.1.1), class member access (5.2.5), or pointer-to-member operation (5.5) whose *cast-expression* is an *id-expression*, s is the set of potential exceptions of the entity selected by the contained *id-expression* (after overload resolution, if applicable).

(14.1.2) — Otherwise, if the *postfix-expression* has type "noexcept function" or "pointer to noexcept function", s is the empty set.

(14.1.3) — Otherwise, s is the set of all types contains the pseudo-type "any".

(14.2) — If e implicitly invokes a function (such as an overloaded operator, an allocation function in a *new-expression*, or a destructor if e is a full-expression (1.9)), s is the set of potential exceptions of the function.

(14.3) — If e initializes an object of type D using an inherited constructor for a class of type B (12.6.3), s also contains the sets of potential exceptions of the implied constructor invocations for subobjects of D that are not subobjects of B (including default argument expressions used in such invocations) as selected by overload resolution, and the sets of potential exceptions of the initialization of non-static data members from *brace-or-equal-initializers* (12.6.2).

(14.4) — if e is a *throw-expression* (5.17), s is the set of all types consists of the type of the exception object that would be initialized by the operand, if present, or the pseudo-type "any" otherwise.

(14.5) — if e is a *dynamic_cast* expression that casts to a reference type and requires a run-time check (5.2.7), s is the set of all types consists of the type `std::bad_cast`.

(14.6) — if e is a *typeid* expression applied to a glvalue expression whose type is a polymorphic class type (5.2.8), s is the set of all types consists of the type `std::bad_typeid`.

(14.7) — if e is a *new-expression* with a non-constant *expression* in the *nopt-new-declarator* (5.3.4), s is the set of all types consists of the type `std::bad_array_new_length`.

[Example: Given the following declarations

```
void f() throw(int)noexcept(false);
void g();
struct A { A(); };
struct B { B() noexcept; };
struct D { D() throw(double)noexcept(false); };
```

the set of potential exceptions for some sample expressions is:

(14.87.1) — for $f()$, the set is the set of all types consists of `int`;

(14.97.2) — for $g()$, the set is the set of all types consists of "any";

(14.107.3) — for $\text{new } A$, the set is the set of all types consists of "any";

(14.117.4) — for $B()$, the set is empty;

(14.127.5) — for $\text{new } D$, the set is the set of all types consists of "any" and `double`.

— end example]

15 A function with an implied non-throwing exception specification, where the function's type is declared to be τ , is instead considered to be of type "noexcept τ ".

16 Given an implicitly declared special member function f of some class x , the set of potential exceptions of the implicitly-declared special member function f is considered to have an implicit exception specification that consists of all the members from

the following sets:

(16.1) — if f is a constructor,

(16.1.1) — the sets of potential exceptions of the constructor invocations

(16.1.1.1) — for x 's non-variant non-static data members,

(16.1.1.2) — for x 's direct base classes, and

(16.1.1.3) — if x is non-abstract (10.4), for x 's virtual base classes,

(including default argument expressions used in such invocations) as selected by overload resolution for the implicit definition of f (12.1). [*Note:* Even though destructors for fully-constructed subobjects are invoked when an exception is thrown during the execution of a constructor (15.2), their exception specifications do not contribute to the exception specification of the constructor, because an exception thrown from such a destructor could never escape the constructor (15.1, 15.5.1). — *end note*]

(16.1.2) — the sets of potential exceptions of the initialization of non-static data members from *brace-or-equal-initializers* that are not ignored (12.6.2);

(16.2) — if f is an assignment operator, the sets of potential exceptions of the assignment operator invocations for x 's non-variant non-static data members and for x 's direct base classes (including default argument expressions used in such invocations), as selected by overload resolution for the implicit definition of f (12.8);

(16.3) — if f is a destructor, the sets of potential exceptions of the destructor invocations for x 's non-variant non-static data members and for x 's virtual and direct base classes.

17 An implicitly-declared special member function (Clause 12) is considered to have an implicit exception specification, as follows, where s is the set of potential exceptions of the implicitly-declared member function:

(17.1) — if s contains the pseudo-type "any", the implicit exception specification is the set of all types;

(17.2) — otherwise, the implicit exception specification contains all the types in s .

17 [*Note:* An instantiation of an inheriting constructor template has an implied exception specification as if it were a non-template inheriting constructor. — *end note*]

[*Example:*

```
struct A {
    A(int = (A(5), 0)) noexcept;
    A(const A&) throw();
    A(A&&) throw();
    ~A() throw(X);
};
struct B {
    B() throw();
    B(const B&) = default; // exception specification contains no types
    B(B&&, int = (throw Y(), 0)) noexcept;
    ~B() noexcept(false)throw(Y);
};
int n = 7;
struct D : public A, public B {
    int * p = new (std::nothrow) int[n];
    // exception specification of D::D() contains the set of all typesX-and-std::bad_array_new_length
    // exception specification of D::D(const D&) contains no types
    // exception specification of D::D(D&&) contains the set of all typesY
    // exception specification of D::~D() contains the set of all typesX-and-Y
};
```

Furthermore, if $A::\sim A()$ ~~or $B::\sim B()$~~ were virtual, the exception specification of $D::\sim D()$ would not be as restrictive as that of $A::\sim A$, and the program would be ill-formed since a function that overrides a virtual function from a base class shall have an *exception-specification* at least as restrictive as that in the base class. — *end example*]

18 A deallocation function (3.7.4.2) with no explicit *exception-specification* has an exception specification that is the empty set.

19 An *exception-specification* is considered to be *needed* when:

(19.1) — in an expression, the function is the unique lookup result or the selected member of a set of overloaded functions (3.4, 13.3, 13.4);

(19.2) — the function is odr-used (3.2) or, if it appears in an unevaluated operand, would be odr-used if the expression were

potentially-evaluated;

(19.3) — the *exception-specification* is compared to that of another declaration (e.g., an explicit specialization or an overriding virtual function);

(19.4) — the function is defined; or

(19.5) — the *exception-specification* is needed for a defaulted special member function that calls the function. [*Note:* A defaulted declaration does not require the *exception-specification* of a base member function to be evaluated until the implicit *exception-specification* of the derived function is needed, but an explicit *exception-specification* needs the implicit *exception-specification* to compare against. — *end note*]

The *exception-specification* of a defaulted special member function is evaluated as described above only when needed; similarly, the *exception-specification* of a specialization of a function template or member function of a class template is instantiated only when needed.

~~20 In a *dynamic-exception-specification*, a *type-id* followed by an ellipsis is a *pack expansion* (14.5.3).~~

~~21 [*Note:* The use of *dynamic-exception-specifications* is deprecated (see Annex D). — *end note*]~~

15.5 Special functions [except.special]

1 The functions `std::terminate()` (15.5.1) ~~and `std::unexpected()` (15.5.2) are~~ is used by the exception handling mechanism for coping with errors related to the exception handling mechanism itself. The function `std::current_exception()` (18.8.5) and the class `std::nested_exception` (18.8.6) can be used by a program to capture the currently handled exception.

15.5.1 The `std::terminate()` function [except.terminate]

1 In some situations exception handling must be abandoned for less subtle error handling techniques. [*Note:* These situations are:

(1.1) — when the exception handling mechanism, after completing the initialization of the exception object but before activation of a handler for the exception (15.1), calls a function that exits via an exception, or

(1.2) — when the exception handling mechanism cannot find a handler for a thrown exception (15.3), or

(1.3) — when the search for a handler (15.3) encounters the outermost block of a function with an *exception* ~~*noexcept-*~~*specification* that does not allow ~~the~~ exceptions (15.4), or

(1.4) — when the destruction of an object during stack unwinding (15.2) terminates by throwing an exception, or

(1.5) — when initialization of a non-local variable with static or thread storage duration (3.6.3) exits via an exception, or

(1.6) — when destruction of an object with static or thread storage duration exits via an exception (3.6.4), or

(1.7) — when execution of a function registered with `std::atexit` or `std::at_quick_exit` exits via an exception (18.5), or

(1.8) — when a *throw-expression* (5.17) with no operand attempts to rethrow an exception and no exception is being handled (15.1), or

~~(1.9) — when `std::unexpected` exits via an exception of a type that is not allowed by the previously violated exception specification, and `std::bad_exception` is not included in that exception specification (15.5.2), or~~

~~(1.10) — when the implementation[™]s default `unexpected` exception handler is called (D.5.1), or~~

(1.11) — when the function `std::nested_exception::rethrow_nested` is called for an object that has captured no exception (18.8.7), or

(1.12) — when execution of the initial function of a thread exits via an exception (30.3.1.2), or

(1.13) — when the destructor or the copy assignment operator is invoked on an object of type `std::thread` that refers to a joinable thread (30.3.1.3, 30.3.1.4), or

(1.14) — when a call to a `wait()`, `wait_until()`, or `wait_for()` function on a condition variable (30.5.1, 30.5.2) fails to meet a postcondition.

— *end note*]

2 In such cases, `std::terminate()` is called (18.8.4). In the situation where no matching handler is found, it is implementation-defined whether or not the stack is unwound before `std::terminate()` is called. In the situation where the search for a handler (15.3) encounters the outermost block of a function with an *exception-specification* that does not allow the exceptions (15.4), it is implementation-defined whether the stack is unwound, unwound partially, or not unwound at all before `std::terminate()` is called. In all other situations, the stack shall not be unwound before `std::terminate()` is called. An implementation is not permitted to finish stack unwinding prematurely based on a determination that the unwind process will eventually cause a call to `std::terminate()`.

15.5.2 The `std::unexpected()` function [except.unexpected]

1 If a function with a *dynamic-exception-specification* exits via an exception of a type that is not allowed by its exception specification, the function `std::unexpected()` is called (D.5) immediately after completing the stack unwinding for the former function.

2 [Note: By default, `std::unexpected()` calls `std::terminate()`, but a program can install its own handler function (D.5.2). In either case, the constraints in the following paragraph apply. —end note]

3 The `std::unexpected()` function shall not return, but it can throw (or rethrow) an exception. If it throws a new exception which is allowed by the exception specification which previously was violated, then the search for another handler will continue at the call of the function whose exception specification was violated. If it exits via an exception of a type that the *dynamic-exception-specification* does not allow, then the following happens: If the *dynamic-exception-specification* does not include the class `std::bad_exception` (18.8.2) then the function `std::terminate()` is called, otherwise the thrown exception is replaced by an implementation-defined object of type `std::bad_exception` and the search for another handler will continue at the call of the function whose *dynamic-exception-specification* was violated.

4 [Note: Thus, a *dynamic-exception-specification* guarantees that a function exits only via an exception of one of the listed types. If the *dynamic-exception-specification* includes the type `std::bad_exception` then any exception type not on the list may be replaced by `std::bad_exception` within the function `std::unexpected()`. —end note]

17.6.4.7 Handler functions [handler.functions]

1 The C++ standard library provides default versions of the following handler functions (Clause 18):

(1.1) — `unexpected_handler`

(1.2) — `terminate_handler`

2 A C++ program may install different handler functions during execution, by supplying a pointer to a function defined in the program or the library as an argument to (respectively):

(2.1) — `set_new_handler`

(2.2) — `set_unexpected`

(2.3) — `set_terminate`

See also: subclauses 18.6.3, Storage allocation errors, and 18.8, Exception handling.

3 A C++ program can get a pointer to the current handler function by calling the following functions:

(2.1) — `get_new_handler`

(2.2) — `get_unexpected`

(2.3) — `get_terminate`

4 Calling the `set_*` and `get_*` functions shall not incur a data race. A call to any of the `set_*` functions shall synchronize with subsequent calls to the same `set_*` function and to the corresponding `get_*` function.

17.6.4.8 Other functions [res.on.functions]

(2.2) — for handler functions (18.6.3.3, 18.8.4.1, ~~D.5.1~~), if the installed handler function does not implement the semantics of the applicable *Required behavior*: paragraph.

17.6.5.12 Restrictions on exception handling [res.on.exception.handling]

1 Any of the functions defined in the C++ standard library can report a failure by throwing an exception of a type described in its

Throws: paragraph. ~~An implementation may strengthen the exception specification for a non-virtual function by adding a non-throwing *noexcept-specification*.~~

2 A function may throw an object of a type not listed in its **Throws** clause if its type is derived from a type named in the **Throws** clause and would be caught by an exception handler for the base type.

3 Functions from the C standard library shall not throw exceptions¹⁸⁸ except when such a function calls a program-supplied function that throws an exception.¹⁸⁹

4 Destructor operations defined in the C++ standard library shall not throw exceptions. Every destructor in the C++ standard library shall behave as if it had a non-throwing exception specification. Any other functions defined in the C++ standard library that do not have an *exception-specification* may throw implementation-defined exceptions unless otherwise specified.¹⁹⁰ ~~An implementation may strengthen this implicit *exception-specification* by adding an explicit one.~~¹⁹¹

5 An implementation may strengthen the exception specification for a non-virtual function by adding an *exception-specification* that does not allow exceptions.

~~191) That is, an implementation may provide an explicit *exception-specification* that defines the subset of "any" exceptions thrown by that function. This implies that the implementation may list implementation-defined types in such an *exception-specification*.~~

18.8 Exception Handling [support.exception]

1 The header <exception> defines several types and functions related to the handling of exceptions in a C++ program.

Header <exception> synopsis

```
namespace std {
    class exception;
    class bad_exception;
    class nested_exception;

    typedef void (*unexpected_handler)();
    unexpected_handler get_unexpected() noexcept;
    unexpected_handler set_unexpected(unexpected_handler f) noexcept;
    [[noreturn]] void unexpected();

    typedef void (*terminate_handler)();
    terminate_handler get_terminate() noexcept;
    terminate_handler set_terminate(terminate_handler f) noexcept;
    [[noreturn]] void terminate() noexcept;

    int uncaught_exceptions() noexcept;
    // D.9X, uncaught_exception (deprecated)
    bool uncaught_exception() noexcept;

    typedef unspecified exception_ptr;

    exception_ptr current_exception() noexcept;
    [[noreturn]] void rethrow_exception(exception_ptr p);
    template <class E> exception_ptr make_exception_ptr(E e) noexcept;
    template <class T> [[noreturn]] void throw_with_nested(T&& t);
    template <class E> void rethrow_if_nested(const E& e);
}
```

18.8.3 Class bad_exception [bad.exception]

1 The class bad_exception defines the type of objects referenced by the exception_ptr returned from a call to current_exception (18.8.6 [propagation]) when the currently active exception object fails to copy~~thrown as described in (15.5.2).~~

23.3.7.8 Zero sized arrays [array.zero]

4 Member function swap() shall have an *exception-specification* that does not allow exceptions ~~a *noexcept-specification* which is equivalent to noexcept(true).~~

Annex B (informative) Implementation quantities [implimits]

~~(2.40) — Throw specifications on a single function declaration [256].~~

C.4.x Clause 15: declarations [diff.cpp14.dcl.dcl]

15.4

Change: Remove dynamic exception specifications.

Rationale: Dynamic exception specifications were a deprecated feature that were complex and brittle in use. They interacted badly with the type system, which would be a more significant issue in C++17 where exception specifications become part of the function type

Effect on original feature: A valid C++ 2014 function declaration, member-function-declaration, function-pointer-declaration, or function-reference definition, which has a potentially throwing dynamic exception specification will be rejected as ill-formed in this international standard. Violating a non-throwing dynamic exception specification will call `terminate` rather than `unexpected` and might not perform stack unwinding prior to such a call.

D.2 Dynamic exception specifications [depr.except.spec]

1 The exception-specification `throw()` use of dynamic exception specifications is deprecated.

~~D.5 Violating exception-specifications [exception.unexpected]~~

~~D.5.1 Type `unexpected_handler` [unexpected.handler]~~

```
typedef void (*unexpected_handler)();
```

~~1 The type of a *handler function* to be called by `unexpected()` when a function attempts to throw an exception not listed in its *dynamic exception specification*.~~

~~2 *Required behavior:* An `unexpected_handler` shall not return. See also 15.5.2.~~

~~3 *Default behavior:* The implementation's default `unexpected_handler` calls `std::terminate()`.~~

~~D.5.2 `set_unexpected` [set.unexpected]~~

```
unexpected_handler set_unexpected(unexpected_handler f) noexcept;
```

~~1 *Effects:* Establishes the function designated by `f` as the current `unexpected_handler`.~~

~~2 *Remark:* It is unspecified whether a null pointer value designates the default `unexpected_handler`.~~

~~3 *Returns:* The previous `unexpected_handler`.~~

~~D.5.3 `get_unexpected` [get.unexpected]~~

```
unexpected_handler get_unexpected() noexcept;
```

~~1 *Returns:* The current `unexpected_handler`. [*Note:* This may be a null pointer value. — *end note*]~~

~~D.5.4 `unexpected` [unexpected]~~

```
[[noreturn]] void unexpected();
```

~~1 *Remarks:* Called by the implementation when a function exits via an exception not allowed by its *exception specification* (15.5.2), in effect after evaluating the *throw-expression* (D.5.1). May also be called directly by the program.~~

~~2 *Effects:* Calls the current `unexpected_handler` function. [*Note:* A default `unexpected_handler` is always considered a callable handler in this context. — *end note*]~~

Second cut: simplified wording

In addition to the previously proposed edits, make the following changes, and completely replace clause 15.4 [except.spec] with the text below.

Drafting notes:

- Still missing Annex C: missing compatibility clause

5.3.7 noexcept operator [expr.unary.noexcept]

1 The `noexcept` operator determines whether the evaluation of its operand, which is an unevaluated operand (Clause 5), can throw an exception (15.1).

noexcept-expression:

`noexcept ( expression )`

2 The result of the `noexcept` operator is a constant of type `bool` and is a prvalue.

3 The result of the `noexcept` operator is `true` unless the expression allows exceptions (15.4) ~~if the set of potential exceptions of the expression (15.4) is empty, and `false` otherwise.~~

8.4.2 Explicitly-defaulted functions [dcl.fct.def.default]

3 If a function that is explicitly defaulted is declared with an *exception-specification* that allows exceptions that are not allowed by ~~is not compatible (15.4) with~~ the exception specification of the implicit declaration, then

(3.1) — if the function is explicitly defaulted on its first declaration, it is defined as deleted;

(3.2) — otherwise, the program is ill-formed.

15.4 Exception specifications [except.spec]

1 The *exception-specification* of a function is a (possibly empty) set of types, indicating that the function might exit via an exception that matches a handler of one of the types in the set; the (conceptual) set of all types is used to denote that the function might exit via an exception of arbitrary type. If the set is empty, the function is said to have a *non-throwing exception specification*. The exception specification is either defined explicitly by using an *exception-specification* as a suffix of a function declaration's declarator (8.3.5) or implicitly.

exception-specification:

dynamic-exception-specification

noexcept-specification

dynamic-exception-specification:

`throw ( type-id-listopt )`

type-id-list:

`type-id ...opt`

`type-id-list, type-id ...opt`

noexcept-specification:

`noexcept ( constant-expression )`

`noexcept`

In a *noexcept-specification*, the *constant-expression*, if supplied, shall be a constant expression (5.20) that is contextually converted to `bool` (Clause 4). A `(` token that follows `noexcept` is part of the *noexcept-specification* and does not commence an initializer (8.5).

2 A type denoted in a *dynamic-exception-specification* shall not denote an incomplete type or an rvalue reference type. A type denoted in a *dynamic-exception-specification* shall not denote a pointer or reference to an incomplete type, other than `â€œpointer to cv voidâ€`. A type `cv T` denoted in a *dynamic-exception-specification* is adjusted to type `T`. A type `â€œarray of Tâ€`, or function type `T` denoted in a *dynamic-exception-specification* is adjusted to type `â€œpointer to Tâ€`. A *dynamic-exception-specification* denotes an exception specification that is the set of adjusted types specified thereby.

3 The *exception-specification* `noexcept` or `noexcept (constant-expression)`, where the *constant-expression* yields `true`, denotes an exception specification that is the empty set. The *exception-specification* `noexcept (constant-expression)`, where the *constant-expression* yields `false`, or the absence of an *exception-specification* in a function declarator other than that for a destructor (12.4) or a deallocation function (3.7.4.2) denotes an exception specification that is the set of all types.

4 Two *exception-specifications* are *compatible* if the sets of types they denote are the same.

5 If any declaration of a function has an *exception-specification* that is not a *noexcept-specification* allowing all exceptions, all declarations, including the definition and any explicit specialization, of that function shall have a compatible *exception-specification*. If any declaration of a pointer to function, reference to function, or pointer to member function has an *exception-specification*, all occurrences of that declaration shall have a compatible *exception-specification*. If a declaration of a function has an implicit exception specification, other declarations of the function shall not specify an *exception-specification*. In an explicit

instantiation an *exception-specification* may be specified, but is not required. If an *exception-specification* is specified in an explicit instantiation directive, it shall be compatible with the *exception-specifications* of other declarations of that function. A diagnostic is required only if the *exception-specifications* are not compatible within a single translation unit.

6 If a virtual function has an exception specification, all declarations, including the definition, of any function that overrides that virtual function in any derived class shall only allow exceptions that are allowed by the exception specification of the base class virtual function, unless the overriding function is defined as deleted. [*Example:*

```
struct B {
    virtual void f() throw (int, double);
    virtual void g();
};

struct D: B {
    void f(); // ill-formed
    void g() throw (int); // OK
};
```

The declaration of `D::f` is ill-formed because it allows all exceptions, whereas `B::f` allows only `int` and `double`. — *end example*]

7 An *exception-specification* can include the same type more than once and can include classes that are related by inheritance, even though doing so is redundant. [*Note:* An *exception-specification* can also include the class `std::bad_exception` (18.8.3). — *end note*]

8 A function is said to allow an exception of type `E` if its exception specification contains a type `T` for which a handler of type `T` would be a match (15.3) for an exception of type `E`. A function is said to allow all exceptions if its exception specification is the set of all types.

9 Whenever an exception of type `E` is thrown and the search for a handler (15.3) encounters the outermost block of a function with an exception specification that does not allow `E`, then

- (9.1) — if the function definition has a *dynamic-exception-specification*, the function `std::unexpected()` is called (15.5.2);
- (9.2) — otherwise, the function `std::terminate()` is called (15.5.1).

[*Example:*

```
class X { };
class Y { };
class Z: public X { };
class W { };

void f() throw (X, Y) {
    int n = 0;
    if (n) throw X(); // OK
    if (n) throw Z(); // also OK
    throw W(); // will call std::unexpected()
}
```

— *end example*]

[*Note:* A function can have multiple declarations with different non-throwing *exception-specifications*; for this purpose, the one on the function definition is used. — *end note*]

10 An implementation shall not reject an expression merely because when executed it throws or might throw an exception that the containing function does not allow. [*Example:*

```
extern void f() throw(X, Y);
void g() throw(X) {
    f(); // OK
}
```

the call to `f` is well-formed even though when called, `f` might throw exception `Y` that `g` does not allow. — *end example*]

11 [*Note:* An exception specification is not considered part of a function’s type; see 8.3.5. — *end note*]

12 A *potential-exception* of a given context is either a type that might be thrown as an exception or a pseudo-type, denoted by "any", that represents the situation where an exception of an arbitrary type might be thrown. A subexpression `e1` of an expression `e` is an immediate subexpression if there is no subexpression `e2` of `e` such that `e1` is a subexpression of `e2`.

13 The *set of potential exceptions of a function, function pointer, or member function pointer* `f` is defined as follows:

- (13.1) — If the exception specification of `f` is the set of all types, the set consists of the pseudo-type "any".

(13.2) — Otherwise, the set consists of every type in the exception specification of f .

14 The *set of potential exceptions of an expression* e is empty if e is a core constant expression (5.20). Otherwise, it is the union of the sets of potential exceptions of the immediate subexpressions of e , including default argument expressions used in a function call, combined with a set s defined by the form of e , as follows:

(14.1) — If e is a function call (5.2.2):

(14.1.1) — If its *postfix-expression* is a (possibly parenthesized) *id-expression* (5.1.1), class member access (5.2.5), or pointer-to-member operation (5.5) whose *cast-expression* is an *id-expression*, s is the set of potential exceptions of the entity selected by the contained *id-expression* (after overload resolution, if applicable):

(14.1.2) — Otherwise, if the *postfix-expression* has type "noexcept function" or "pointer to noexcept function", s is the empty set.

(14.1.3) — Otherwise, s contains the pseudo-type "any".

(14.2) — If e implicitly invokes a function (such as an overloaded operator, an allocation function in a *new-expression*, or a destructor if e is a full-expression (1.9)), s is the set of potential exceptions of the function:

(14.3) — If e initializes an object of type D using an inherited constructor for a class of type B (12.6.3), s also contains the sets of potential exceptions of the implied constructor invocations for subobjects of D that are not subobjects of B (including default argument expressions used in such invocations) as selected by overload resolution, and the sets of potential exceptions of the initialization of non-static data members from *brace-or-equal-initializers* (12.6.2).

(14.4) — if e is a *throw-expression* (5.17), s consists of the type of the exception object that would be initialized by the operand, if present, or the pseudo-type "any" otherwise.

(14.5) — if e is a `dynamic_cast` expression that casts to a reference type and requires a run-time check (5.2.7), s consists of the type `std::bad_cast`:

(14.6) — if e is a *typeid* expression applied to a glvalue expression whose type is a polymorphic class type (5.2.8), s consists of the type `std::bad_typeid`:

(14.7) — if e is a *new-expression* with a non-constant *expression* in the *nopt-new-declarator* (5.3.4), s consists of the type `std::bad_array_new_length`:

[*Example:* Given the following declarations

```
void f() throw(int);
void g();
struct A { A(); };
struct B { B() noexcept; };
struct D { D() throw (double); };
```

the set of potential exceptions for some sample expressions is:

(14.8) — for $f()$, the set consists of `int`;

(14.9) — for $g()$, the set consists of "any";

(14.10) — for `new A`, the set consists of "any";

(14.11) — for $B()$, the set is empty;

(14.12) — for `new D`, the set consists of "any" and `double`.

— *end example* }

15 A function with an implied non-throwing exception specification, where the function^{â€™s} type is declared to be τ , is instead considered to be of type "noexcept τ ".

16 Given an implicitly declared special member function f of some class X , the *set of potential exceptions of the implicitly-declared special member function* f consists of all the members from the following sets:

(16.1) — if f is a constructor,

(16.1.1) — the sets of potential exceptions of the constructor invocations

(16.1.1.1) — for X 's non-variant non-static data members,

(16.1.1.2) — for x 's direct base classes, and

(16.1.1.3) — if x is non-abstract (10.4), for x 's virtual base classes;

(including default argument expressions used in such invocations) as selected by overload resolution for the implicit definition of f (12.1). [*Note:* Even though destructors for fully-constructed subobjects are invoked when an exception is thrown during the execution of a constructor (15.2), their exception specifications do not contribute to the exception specification of the constructor, because an exception thrown from such a destructor could never escape the constructor (15.1, 15.5.1). — *end note*]

(16.1.2) — the sets of potential exceptions of the initialization of non-static data members from *brace-or-equal-initializers* that are not ignored (12.6.2);

(16.2) — if f is an assignment operator, the sets of potential exceptions of the assignment operator invocations for x 's non-variant non-static data members and for x 's direct base classes (including default argument expressions used in such invocations), as selected by overload resolution for the implicit definition of f (12.8);

(16.3) — if f is a destructor, the sets of potential exceptions of the destructor invocations for x 's non-variant non-static data members and for x 's virtual and direct base classes;

17 An implicitly-declared special member function (Clause 12) is considered to have an implicit exception specification, as follows, where s is the set of potential exceptions of the implicitly-declared member function:

(17.1) — if s contains the pseudo-type "any", the implicit exception specification is the set of all types;

(17.2) — otherwise, the implicit exception specification contains all the types in s .

[*Note:* An instantiation of an inheriting constructor template has an implied exception specification as if it were a non-template inheriting constructor. — *end note*]

[*Example:*

```
struct A {
    A(int = (A(5), 0)) noexcept;
    A(const A&) throw();
    A(A&) throw();
    ~A() throw(X);
};
struct B {
    B() throw();
    B(const B&) = default; // exception specification contains no types
    B(B&, int = (throw Y(), 0)) noexcept;
    ~B() throw(Y);
};
int n = 7;
struct D : public A, public B {
    int *p = new (std::nothrow) int[n];
    // exception specification of D::D() contains X and std::bad_array_new_length
    // exception specification of D::D(const D&) contains no types
    // exception specification of D::D(D&) contains Y
    // exception specification of D::~D() contains X and Y
};
```

Furthermore, if $A::~A()$ or $B::~B()$ were virtual, $D::~D()$ would not be as restrictive as that of $A::~A$, and the program would be ill-formed since a function that overrides a virtual function from a base class shall have an *exception specification* at least as restrictive as that in the base class. — *end example*]

18 A deallocation function (3.7.4.2) with no explicit *exception specification* has an exception specification that is the empty set.

19 An *exception specification* is considered to be *needed* when:

(19.1) — in an expression, the function is the unique lookup result or the selected member of a set of overloaded functions (3.4, 13.3, 13.4);

(19.2) — the function is odr-used (3.2) or, if it appears in an unevaluated operand, would be odr-used if the expression were potentially-evaluated;

(19.3) — the *exception specification* is compared to that of another declaration (e.g., an explicit specialization or an overriding virtual function);

(19.4) — the function is defined; or

(19.5) — the *exception specification* is needed for a defaulted special member function that calls the function. [*Note:* A defaulted

declaration does not require the *exception-specification* of a base member function to be evaluated until the implicit *exception-specification* of the derived function is needed, but an explicit *exception-specification* needs the implicit *exception-specification* to compare against. — *end note* }

The *exception-specification* of a defaulted special member function is evaluated as described above only when needed; similarly, the *exception-specification* of a specialization of a function template or member function of a class template is instantiated only when needed.

20 In a *dynamic-exception-specification*, a *type-id* followed by an ellipsis is a pack expansion (14.5.3).

21 [*Note:* The use of *dynamic-exception-specifications* is deprecated (see Annex D). — *end note*]

1 The *exception specification* of a function is a predicate indicating whether the function might exit via an exception. If the predicate evaluates as `false`, the function is said to have a *non-throwing exception specification*. The exception specification is either defined explicitly by using an *exception-specification* as a suffix of a function declaration's declarator (8.3.5) or implicitly.

```
exception-specification:  
noexcept ( constant-expression )  
noexcept  
throw ( )
```

2 In an *exception-specification*, the *constant-expression*, if supplied, shall be a constant expression (5.20) that is contextually converted to `bool` (Clause 4). A `(` token that follows `noexcept` is part of the *exception-specification* and does not commence an initializer (8.5). The *exception-specification* `noexcept` (without a *const-expression*) is equivalent to the *exception-specification* `noexcept(true)`. The *exception-specification* `throw()` is deprecated (see Annex D), and equivalent to the *exception-specification* `noexcept(true)`.

3 An *exception specification* is said to allow exceptions if the predicate evaluates to `false`. Otherwise, it does not allow exceptions.

4 If any declaration of a function has an *exception specification* that does not allow exceptions, all declarations, including the definition and any explicit specialization, of that function shall have an *exception specification* that does not allow exceptions. If any declaration of a pointer to function, reference to function, or pointer to member function has an *exception specification* that does not allow exceptions, all occurrences of that declaration shall have an *exception-specification* that does not allow exceptions. If a declaration of a function has an implicit exception specification, other declarations of the function shall not specify an *exception-specification*. In an explicit instantiation an *exception-specification* may be specified, but is not required. If an *exception-specification* is specified in an explicit instantiation directive, it shall allow exceptions only if the *exception-specifications* of all other declarations of that function allow exceptions. A diagnostic is required only if the *exception-specifications* are not compatible within a single translation unit.

5 If a virtual function has an exception specification that does not allow exceptions, all declarations, including the definition, of any function that overrides that virtual function in any derived class shall not allow exceptions, unless the overriding function is defined as deleted. [*Example:*

```
struct B {  
    virtual void f() noexcept;  
    virtual void g();  
    virtual void h() noexcept;  
};  
  
struct D: B {  
    void f(); // ill-formed  
    void g() noexcept; // OK  
    void h() = delete; // OK  
};
```

The declaration of `D::f` is ill-formed because it allows all exceptions, whereas `B::f` allows no exceptions. — *end example*]

6 Whenever an exception is thrown and the search for a handler (15.3) encounters the outermost block of a function with an exception specification that does not allow exceptions, the function `std::terminate()` is called (15.5.1).

7 An implementation shall not reject an expression merely because when executed it throws or might throw an exception that the containing function does not allow. [*Example:*

```
extern void f() noexcept;  
void g() {  
    f(); // OK  
}
```

the call to `f` is well-formed even though, when called, `f` might throw an exception that `g` does not allow. — *end example*]

8 An expression allows an exception if and only if any of its immediate subexpressions allows an exception. A subexpression `e1`

of an expression *e* is an immediate subexpression if there is no subexpression *e*₂ of *e* such that *e*₁ is a subexpression of *e*₂.

9 If *e* initializes an object of type *D* using an inherited constructor for a class of type *B* (12.6.3), the set of immediate subexpressions of *e* includes the implied constructor invocations for subobjects of *D* that are not subobjects of *B* (including default argument expressions used in such invocations) as selected by overload resolution, and the initialization of non-static data members from *brace-or-equal-initializers* (12.6.2).

10 An expression (or subexpression) *e* does not allow exceptions unless

(10.1) *e* invokes a function, or member function, whose *exception specification* allows exceptions, or

(10.2) *e* implicitly invokes a function (such as an overloaded operator, an allocation function in a *new-expression*, a constructor for a function argument, or a destructor if *e* is a full-expression (1.9)), that allows exceptions, or

(10.3) *e* is a *throw-expression* (5.17), or

(10.4) *e* is a *dynamic cast* expression that casts to a reference type and requires a run-time check (5.2.7), or

(10.5) *e* is a *typeid* expression applied to a glvalue expression whose type is a polymorphic class type (5.2.8), or

(10.6) *e* is a *new-expression* with a non-constant *expression* in the *nopt-new-declarator* (5.3.4), or

(10.7) any of the immediate subexpressions of *e* allows exceptions.

11 An implicitly declared special member function that is a constructor for a class *x* has an exception specification that does not allow exceptions unless any of the constructors selected by overload resolution in the implicit definition of that constructor to initialize direct base classes and non-static members, and, unless *x* is abstract, to initialize virtual base classes, has an exception specification that allows exceptions, or the invocation of such initialization contains an expression that allows exceptions, such as default argument expressions used in such invocations, or *brace-or-equal-initializers* for non-static data members.

12 [*Note:* Even though destructors for fully-constructed subobjects are invoked when an exception is thrown during the execution of a constructor (15.2), their exception specifications do not contribute to the exception specification of the constructor, because an exception thrown from such a destructor could never escape the constructor (15.1, 15.5.1). — *end note*]

13 [*Note:* An instantiation of an inheriting constructor template has an implied exception specification as if it were a non-template inheriting constructor. — *end note*]

[*Example:*

```
struct A {
    A(int = (A(5), 0)) noexcept;
    A(const A&) noexcept;
    A(A&&) noexcept;
    ~A();
};
struct B {
    B() throw();
    B(const B&) = default; // implicit exception specification is true
    B(B&&, int = (throw Y(), 0)) noexcept;
    ~B() noexcept(false);
};
int n = 7;
struct D : public A, public B {
    int * p = new (std::nothrow) int[n];
    // D::D() allows exceptions, as the new operator may throw bad array new length
    // D::D(const D&) does not allow exceptions
    // D::D(D&&) allows exceptions, as the default argument for B's constructor may throw
    // D::~~D() allows exceptions
};
```

Furthermore, if *A::~~A()* were virtual, the exception specification of *D::~~D()* would not be as restrictive as that of *A::~~A*, and the program would be ill-formed since a function that overrides a virtual function from a base class shall have an *exception-specification* at least as restrictive as that in the base class. — *end example*]

14 An implicitly declared special member function that is the destructor for a class *x* has an exception specification that does not allow exceptions unless any of the destructors for its direct or virtual base classes and non-variant non-static data members has a destructor with an exception specification that allows exceptions.

15 An implicitly declared special member function *f* that is an assignment operator for a class *x* has an exception specification that does not allow exceptions unless the implicit definition of *f* selects by overload resolution an assignment operator from a direct base class, or from a non-variant non-static data members, that allows exceptions.

16 A function with an implied non-throwing exception specification, where the function's type is declared to be τ , is instead considered to be of type "noexcept τ ".

17 A deallocation function (3.7.4.2) with no explicit *exception-specification* has an exception specification that does not allow exceptions.

18 An *exception-specification* is considered to be *needed* when:

(18.1) — in an expression, the function is the unique lookup result or the selected member of a set of overloaded functions (3.4, 13.3, 13.4);

(18.2) — the function is odr-used (3.2) or, if it appears in an unevaluated operand, would be odr-used if the expression were potentially-evaluated;

(18.3) — the *exception-specification* is compared to that of another declaration (e.g., an explicit specialization or an overriding virtual function);

(18.4) — the function is defined; or

(18.5) — the *exception-specification* is needed for a defaulted special member function that calls the function. [*Note:* A defaulted declaration does not require the *exception-specification* of a base member function to be evaluated until the implicit *exception-specification* of the derived function is needed, but an explicit *exception-specification* needs the implicit *exception-specification* to compare against. — *end note*]

The *exception-specification* of a defaulted special member function is evaluated as described above only when needed; similarly, the *exception-specification* of a specialization of a function template or member function of a class template is instantiated only when needed.

Acknowledgements

Thanks to Jens Maurer for the initial review, identifying a number of proposed edits that had unintended consequences.

VIII. References

- [N0741](#) Exceptions specifications in the Standard Library, Mats Henricson
- [N3051](#) Deprecating Exception Specifications, Doug Gregor
- [N4533](#) Make exception specifications be part of the type system, version 3, Jens Maurer
- [N4086](#) Removing trigraphs?!, Richard Smith
- [N4168](#) Removing `auto_ptr`, Billy Baker
- [N4190](#) Removing `auto_ptr`, `random_shuffle()`, And Old `<functional>` Stuff, Stephan T. Lavavej
- [P0001R0](#) Remove Deprecated Use of the `register` Keyword
- [P0002R0](#) Remove Deprecated `operator++(bool)`
- [P0004R0](#) Remove Deprecated `iostreams` aliases